

NotAndOr - Informe Stage III

1. Introducción

NotAndOr es un DSL para describir y simular circuitos lógicos booleanos secuenciales síncronos. El compilador recibe programas por `stdin`, construye un AST con Flex/Bison, valida reglas semánticas propias del dominio y genera un simulador C99 autocontenido.

Repositorio completo: <https://github.com/tcostamendez/TPE-ATLC>

2. Modelo Computacional

2.1. Dominio

El dominio modela circuitos booleanos de tiempo discreto. Cada señal toma valores `0` o `1`. Los circuitos pueden tener entradas, salidas, cables combinacionales, registros con estado e instancias de otros circuitos.

2.2. Lenguaje

Las construcciones principales son:

- `circuit`: define un módulo reutilizable.
- `input`, `output`, `wire`, `reg`: declaran señales booleanas.
- `=`: define lógica combinacional.
- `<=`: define el próximo valor de un registro.
- `on rising\_edge(signal)` y `on falling\_edge(signal)`: definen actualizaciones ante flancos.
- `and`, `or`, `xor`, `not`: operadores booleanos.
- `.` y `+`: alias de `and` y `or`, respectivamente.
- `CircuitName(...) -> (...)`: instancia un subcircuito con conexiones nombradas.

`clk` no es una palabra reservada. Cualquier señal declarada como `input` puede usarse como reloj de un bloque de flanco.

Código 1: registro de un bit con carga.

```
```txt
circuit Register1 {
input in, load, clk;
output out;
reg value;
wire next;

out = value;
next = (load and in) or ((not load) and value);

on rising_edge(clk) {
value <= next;
}
}
```

### ## 3. Implementación

#### ### 3.1. Frontend

El frontend mantiene la arquitectura Flex/Bison. Stage III agrega el token `falling\_edge` y acepta `.` y `+` como alias léxicos de `and` y `or`. El AST registra el tipo de flanco en cada bloque secuencial.

### ### 3.2. Backend

La fase semántica construye un modelo con tabla global de circuitos y tablas de señales por circuito. Valida nombres únicos, declaraciones, usos, asignabilidad, clocks, interfaces de instancias, outputs determinados y ciclos combinacionales.

La generación de código emite C99 autocontenido. El simulador generado:

- inicializa registros en `0`;
- lee un entero `N`;
- lee `N` filas con los inputs del circuito top en orden de declaración;
- evalúa lógica combinatorial;
- detecta flancos con valores previos de inputs;
- actualiza registros;
- imprime outputs en orden de declaración.

### ### 3.3. Adicionales

Se agregó `--top CircuitName`. Si no se especifica, el top es el último circuito definido.

### ### 3.4. Dificultades Encontradas

La principal dificultad fue separar correctamente la semántica de señales combinacionales y registros sin abandonar la estructura original del template. También fue necesario definir una convención simple de runtime para alimentar estímulos de entrada desde `stdin`.

## ## 4. Futuras Extensiones

- Soporte para buses multibit.
- Inicialización explícita de registros.
- Testbenches declarados en el propio DSL.
- Mejor ordenamiento topológico para generación de lógica combinatorial.
- Reportes de error con ubicación exacta del AST.

## ## 5. Conclusiones

Stage III transforma el proyecto de un frontend sintáctico en un compilador funcional para simulación. La validación semántica captura errores estructurales relevantes del dominio, y la generación C99 permite ejecutar los circuitos sin depender de librerías externas.

## ## 6. Referencias

- Consigna del Proyecto Especial, 12 de marzo de 2026.
- Material de cátedra: Análisis Semántico, Generación de Código y Runtime.

## ## 7. Bibliografía

- Aho, Lam, Sethi y Ullman. Compilers: Principles, Techniques, and Tools.
- IEEE. Standard for Verilog Hardware Description Language.